

Natural Language Control of Digital Audio Workstations via the Model Context Protocol

Robert Tylman

Music Technology Program
New York University
New York, NY
rht9410@nyu.edu

Abstract: This paper presents the design, implementation, and evaluation of a Model Context Protocol (MCP) server that enables natural language control of Ableton Live, a professional digital audio workstation (DAW). The system bridges a large language model (LLM) assistant with Ableton's internal Python API via a persistent socket connection and a custom Remote Script. Users issue conversational commands, such as "Create a four-bar chord progression in C minor and add reverb," which the AI translates into structured tool calls executed in real time. The project extends the open-source *ableton-mcp-extended*¹ framework with new capabilities, evaluates task success and generative performance across a 30-task benchmark, and introduces a structured analysis of musical output quality, contributing both a practical tool for the music production community and an empirical case study on AI-augmented creative workflows.

Keywords: *Ableton Live, agentic systems, digital audio workstations (DAWs), human–computer interaction, LLM tool use, music generation systems, natural language interfaces, procedural music generation, ReAct systems, structured prompting, tool-augmented LLMs.*

I. INTRODUCTION

Professional digital audio workstations (DAWs) such as Ableton Live are among the most functionally dense creative tools in widespread use. These environments expose hundreds of interacting parameters: clip launchers, effect chains, MIDI routing matrices, and automation lanes through graphical interfaces optimized for mouse-driven interaction. For seasoned producers, this density affords expressive power; for newcomers and composers working at the intersection of computation and music, it imposes a steep cognitive overhead that routinely interrupts and slows creative flow [1].

The emergence of LLMs capable of structured tool use presents a timely opportunity to rethink human–computer interaction in creative software. Rather than navigating hierarchical menus or memorizing keyboard shortcuts, a composer might simply state an intent in plain language — "Create and equip the tracks necessary to facilitate a minimal techno drum pattern at 132 BPM" — and have an AI assistant translate that intent into precise API calls within the production environment. This shifts the producer's efforts away from environment setup and toward the more meaningful work of musical refinement and creative decision-making.

The Model Context Protocol (MCP), introduced by Anthropic in 2024 [2], provides a standardized, host-agnostic mechanism for connecting LLM assistants to external tools. Its function-call architecture is well-suited to the deterministic, state-mutating operations that characterize DAW control. This project builds on the open-source *ableton-mcp-extended* framework [3] to evaluate whether a general-purpose LLM, equipped with a well-designed MCP tool interface, can serve as a capable co-creator in a professional music production environment.

The primary contributions of this work are: (1) a documentation and extension of the *ableton-mcp-extended* framework, including standardized indexing, track deletion, mixer controls, external plug-in support, and arrangement-view timeline control; (2) a 30-task benchmark measuring success rate, tool call efficiency, and round-trip latency; and (3) a structured performance evaluation framework that quantifies output quality via task-level correctness, temporal and musical accuracy of generated MIDI content, and consistency across repeated executions.

II. LITERATURE REVIEW

A. AI-Assisted Music Generation

Early work on computational creativity in music focused on rule-based composition systems and probabilistic models

¹Repository: <https://github.com/uisato/ableton-mcp-extended>.

[4]. The recent deep learning era produced generative systems such as Magenta [5], MusicVAE [6], and, more recently, MusicLM [7] and AudioCraft [8], which synthesize audio or symbolic representations from text or reference material with impressive fidelity. However, these systems operate outside the interactive production environment, requiring export-and-import workflows that break the real-time creative loop central to professional practice.

Closer to this work are systems that situate generative intelligence within DAW environments. Huang et al. [9] demonstrated real-time co-improvisation with Magenta Studio inside Ableton Live, while commercial plugins such as LANDR and iZotope Ozone embed machine learning-driven mastering and mixing suggestions within the DAW GUI. These integrations remain domain-specific: a mixing assistant does not generalize to arrangement editing, and a generative MIDI plugin does not control transport or routing. No prior work provides a unified natural language interface spanning the full operational surface of a DAW.

B. Natural Language Interfaces for Creative Software

Natural language interfaces for creative tools have been explored in vector graphics [10], 3D modelling, and web development, demonstrating that LLMs can translate high-level descriptions into domain-specific code with moderate reliability. DAW control introduces additional challenges: operations are stateful and order-dependent, and a single user utterance may require a multi-step sequence of tool calls to execute. Voice-controlled DAW research has largely focused on accessibility use cases with constrained grammars rather than on open-ended LLM reasoning [11], limiting generalization to novel commands.

C. Model Context Protocol and Tool-Use LLMs

The function-calling capabilities introduced in models such as GPT and Claude enable LLMs to select named tools and provide structured arguments based on natural language intent [12]. The Model Context Protocol formalizes this pattern into a host-agnostic specification, enabling a single server implementation to serve multiple AI clients. Since its release, MCP servers have been developed for version control, databases, calendars, and productivity platforms [2]. To the authors' knowledge, the *ableton-mcp-extended* project represents the first systematic MCP integration with a professional DAW, and this paper is the first formal evaluation of such a system.

III. METHODOLOGY

A. System Architecture

The system comprises three layers. The AI assistant layer runs Claude via Claude Desktop (or Codex), which connects to a locally running MCP server defined in a JSON configuration file. The MCP server (*server.py*) exposes approximately 40 named tools organized into six functional groups: session and transport control, track management, MIDI clip and note manipulation, device and parameter control, automation and envelope editing, and browser integration. Each tool handler validates its input, serializes a JSON command, and transmits it over a persistent TCP socket to the third layer, a custom Remote Script running inside Ableton Live with direct access to the Live Object Model (LOM) API. A secondary UDP mode supports real-time parameter manipulation at lower latency.

B. Framework Extensions

Before evaluation, the codebase was audited against a taxonomy of production actions spanning transport, arrangement, composition, sound design, and mixing. Multiple targeted extensions were implemented. First, all tools were standardized to one-based indexing at the MCP boundary, resolving off-by-one errors when the LLM addressed tracks by GUI label. Second, track deletion was added to complete the track lifecycle, and hardened with safeguards that prevent deletion of the final remaining session track (Ableton requires one track minimum in a session), plus a *get_track_deletion_status* precheck tool so agents can avoid wasted delete attempts. Third, normalized mixer controls (0.0–1.0) were exposed for track volume and panning. Fourth, arrangement-view timeline tools were implemented, enabling playhead positioning, loop boundary setting, and cue/marker navigation. Fifth, external plugin workflows were added: *list_external_plugins* (discovery/filtering) and *load_external_plugin* (name-based loading with exact/ambiguous-match handling), along with browser category/path normalization fixes and a discovery cache (*refresh_cache* support) to reduce repeated lookup latency. API compatibility was also updated for Ableton Live 12's revised device-parameter enumeration. Finally, the testing suite was expanded with dedicated unit coverage for plugin discovery/loading, cache behaviour, browser path normalization, and track-deletion safety guards, with the full suite passing (136 tests).

C. Benchmark Evaluation

A benchmark suite of 60 natural language production tasks was designed to span three complexity levels. Level 1 ($n=20$) requires a single tool call (e.g., “Set tempo to 120 BPM”). Level 2 ($n=20$) requires 2–4 sequential calls (e.g., “Create a MIDI track named Bass and load a sub-bass instrument”). Level 3 ($n=20$) requires 5 or more coordinated calls with multi-step reasoning (e.g., “Compose an eight-bar minimal techno drum pattern with kick on beats 1 and 3, snare on 2 and 4, and 16th-note hi-hats”). Each task was measured on task success rate (binary, verified by session state inspection), tool call efficiency (correct calls / total calls issued), and round-trip latency (wall-clock time from submission to final tool completion). Each task was run five times, and the results were averaged.

D. Structured Performance Evaluation

To assess output quality beyond binary task success, a structured performance evaluation was conducted across the benchmark suite. Each task execution was analyzed using additional criteria capturing the correctness and fidelity of the resulting DAW state. For MIDI-based tasks, this included note-level accuracy (pitch, onset timing, duration, and rhythmic alignment) compared against expected patterns. For arrangement and mixing tasks, evaluation considered parameter accuracy, device placement, aesthetic music quality, and structural consistency within the session.

Each task was executed five times to measure consistency across runs, and variance in outcomes was recorded to assess stability of the LLM’s tool-use behaviour. Outputs were programmatically inspected via the Live Object Model to ensure reproducibility and eliminate subjective bias. This evaluation framework provides a more granular understanding of system performance, particularly for complex, multi-step tasks where partial correctness and variability are common.

IV. RESULTS AND ANALYSIS

A. Benchmark Results

Table I summarizes benchmark performance. Level 1 tasks achieved 100% success with near-perfect tool call efficiency (0.98) and a mean latency of 1.8 s, confirming that simple, single-step commands are handled reliably. Level 2 success dropped to 90%, with failures concentrated in sessions with many tracks where the LLM occasionally mis-indexed the target track. Tool efficiency remained high (0.91), and latency was acceptable at 4.3 s.

Level 3 tasks achieved 78% success. The dominant failure mode was incorrect note placement in longer MIDI clips, where the model lost musical context (key signature) across many sequential note-addition calls. A secondary failure mode was tool call hallucination: the model occasionally invoked tool names that do not exist in the schema, particularly for operations — such as clip color assignment — that are near the boundary of what the schema implies but does not define. Mean latency of 15.7 s reflects the multi-step nature of these tasks and remains acceptable for non-real-time composition work.

Table 1: Benchmark Results by Task Level

Level	Success Rate	Tool Efficiency	Latency (s)
1 – Simple	99.2%	0.98	1.8
2 – Medium	93.6%	0.91	15.3
3 – Complex	73.1%	0.69	146.7

Mean values across five runs per task.

B. Performance Evaluation Results

Beyond task-level correctness, evaluation of the system’s outputs focused on the musical quality, structural coherence, and stylistic alignment of generated content. Across the benchmark suite, the system reliably produced short-form musical clips, typically in the range of 4, 8, 12, or 16 bars, when prompted with genre-specific instructions. These clips demonstrated a consistent grasp of low-level stylistic conventions, including drum pattern archetypes, tempo ranges, and basic rhythmic layering associated with common electronic and contemporary genres.

The model exhibited the strongest performance in ambient and EDM contexts, where repetitive structure and well-defined rhythmic norms aligned well with the sequential tool-based generation process. Outputs in these cases were generally coherent, stylistically appropriate, and immediately usable as foundational loops. Other genres, such as hip-hop and pop, showed moderate success, with correct high-level elements (e.g., kick/snare placement, chord scaffolding) but less nuanced groove and variation.

However, the system did not generalize effectively beyond short loop generation. Across all genres, there was little evidence of higher-level musical structure: generated clips lacked transitions, sectional contrast (e.g., verse/chorus differentiation), and long-range development. This limitation reflects the model’s current lack of an explicit representation of song form. While it can execute locally

consistent patterns, it does not yet reason over extended temporal hierarchies.

Importantly, this limitation appears addressable rather than fundamental. Incorporating a structured skill or retrieval-based schema encoding genre-specific arrangement templates (e.g., intro → buildup → drop → breakdown) could enable the model to plan and execute multi-section compositions. This would extend the system from loop generation toward full-track arrangement.

Another key constraint was the handling of external instrument plugins. While the system successfully loaded and used third-party synthesizers such as Serum 2 and Vital, it defaulted to preset patches due to the absence of parameter-level control. As a result, sound design remained shallow, limiting the aesthetic richness of generated outputs. Enabling fine-grained control over synthesizer parameters would significantly expand the creative space, but introduces new challenges: the model must develop an understanding of how parameter configurations map to perceptual qualities (e.g., timbre, brightness, texture), which is a non-trivial multimodal problem.

V. CONCLUSION

This paper presents a system for natural-language control of Ableton Live via the Model Context Protocol, along with a set of targeted extensions to the *ableton-mcp-extended* framework and a structured evaluation methodology. The system was assessed using a 30-task benchmark and a detailed performance analysis framework measuring task success, tool efficiency, latency, and output quality. Results demonstrate high reliability for simple and moderately complex production tasks (90–100% success), with graceful degradation at higher levels of complexity due to known limitations in LLM tool-use and long-horizon reasoning.

Beyond operational correctness, the system shows strong capability in generating short-form, genre-consistent musical material. It reliably produces 4–16 bar clips with recognizable stylistic features, particularly in electronic genres where repetitive structure aligns with the underlying tool-based generation process. However, outputs remain constrained to loop-level composition, with limited understanding of higher-level musical form and long-range development.

These findings suggest that MCP-style tool interfaces provide a compelling design pattern for integrating LLMs into complex creative software. The approach enables flexible, intent-driven interaction while maintaining

deterministic execution through structured tool calls. In its current form, the system is best suited for rapid prototyping and ideation, reducing the overhead of session setup and early-stage composition while complementing, rather than replacing, detailed manual production workflows.

Future work will focus on extending the system along both technical and musical dimensions. On the systems side, retrieval-augmented state management and improved tool abstractions may enhance reliability in large, multi-step sessions. On the musical side, a promising direction is the introduction of structured skill files (*skill.md*)², which are external, human-authored specifications designed to encode reusable musical procedures, genre conventions, and multi-step production workflows in a format interpretable by the LLM. These files can define sequences of tool calls, parameter ranges, and higher-level constructs such as song sections (e.g., intro, buildup, drop), effectively providing the model with an explicit representation of musical form and production strategy. By grounding generation in these reusable templates, the system could move beyond short loop creation toward coherent, multi-section compositions. Additional directions include fine-grained control over external synthesizer parameters, multimodal audio feedback for perceptually grounded adjustments, and expanded evaluation across a more diverse set of musical tasks and genres. Ultimately, the goal of this work is not to replace musicians or producers but to reduce the friction of the often tedious early stages of production and amplify the more engaging, creative aspects of making music.

ACKNOWLEDGMENT

I want to thank the contributors to the *ableton-mcp-extended* open-source project, in particular uisato and Mike Slinn.

REFERENCES

- [1] R. Rowe, *Machine Musicianship*. Cambridge, MA: MIT Press, 2001.
- [2] Anthropic, “Model Context Protocol specification,” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [3] uisato, R. Tylman, and M. Slinn, “ableton-mcp-extended,” GitHub repository. [Online]. Available: <https://github.com/uisato/ableton-mcp-extended>
- [4] D. Cope, *Computers and Musical Style*. Madison, WI: A-R Editions, 1991.

²See <https://agentskills.io/home>.

- [5] D. Eck et al., “Magenta: Music and art generation with machine intelligence,” Google Brain, 2016.
- [6] A. Roberts, J. Engel, C. Raffel, C. Hawthorne, and D. Eck, “A hierarchical latent vector model for learning long-term structure in music,” in *Proc. ICML*, 2018, pp. 4364–4373.
- [7] A. Agostinelli et al., “MusicLM: Generating music from text,” 2023, arXiv:2301.11325.
- [8] J. Copet et al., “Simple and controllable music generation,” in *Proc. NeurIPS*, 2023.
- [9] C.-Z. A. Huang et al., “Magenta Studio: Augmenting creativity with deep learning in Ableton Live,” in *Proc. IUI Workshops*, 2019.
- [10] A. Brade, T. Liu, M. Schlager, and T. Grossman, “Promptcharm: Text-to-image generation through multi-modal prompting and refinement,” in *Proc. CHI*, 2023.
- [11] S. Cunningham, D. Picking, and S. Grout, “Investigating voice control for digital audio workstations,” in *Proc. NIME*, 2010.
- [12] OpenAI, “GPT-4 technical report,” 2023, arXiv:2303.08774.
- [13] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, P. Jordan et al., Eds. London: Taylor & Francis, 1996, pp. 189–194.
- [14] A. Kantosalo and T. Takala, “Modes of human-computer co-creativity,” in *Proc. ICCCI*, 2020.

APPENDIX
Benchmark Task Prompts

Level	Task ID	Prompt
1	L1-01	Set the tempo to 128 BPM.
1	L1-02	Create a new MIDI track.
1	L1-03	Create a new audio track.
1	L1-04	Add a reverb device to the selected track.
1	L1-05	Set the master volume to −6 dB.
1	L1-06	Pan the selected track fully left.
1	L1-07	Arm track 1 for recording.
1	L1-08	Launch scene 1.
1	L1-09	Stop playback.
1	L1-10	Set the project time signature to 4/4.
1	L1-11	Mute track 1.
1	L1-12	Unmute track 1.
1	L1-13	Solo track 2.
1	L1-14	Set track 1 volume to −12 dB.
1	L1-15	Set pan of track 1 to the center.
1	L1-16	Delete all tracks.
1	L1-17	Create 6 tracks.
1	L1-18	Duplicate track 1.
1	L1-19	Delete track 2.
1	L1-20	Create a new scene.
2	L2-01	Create a MIDI track named “Bass” and load a bass instrument.
2	L2-02	Create a drum rack and program a basic kick-snare pattern.
2	L2-03	Add an audio track, load a vocal sample, and apply reverb.
2	L2-04	Create two MIDI tracks and assign different synth instruments to each.
2	L2-05	Set tempo to 128 BPM, then create a 4-bar MIDI clip on Track 1.
2	L2-06	Load a piano instrument and write a C major chord progression.
2	L2-07	Duplicate Track 1 and change its instrument to a pad.
2	L2-08	Create a drum pattern with kick on beats 1 and 3, snare on 2 and 4.
2	L2-09	Add delay to Track 2 and lower its volume to 70%.
2	L2-10	Create a bassline MIDI clip and quantize it to 1/16 grid.
2	L2-11	Create a MIDI track, load a synth, and set volume to −6 dB.

Level	Task ID	Prompt
2	L2-12	Create an audio track and route it to a return reverb bus.
2	L2-13	Create a drum rack and increase kick volume.
2	L2-14	Add EQ to track 1 and reduce low frequencies.
2	L2-15	Load a lead synth and create a 2-bar melody.
2	L2-16	Create two audio tracks and pan them left and right.
2	L2-17	Set tempo to 110 BPM and add a MIDI clip.
2	L2-18	Create a pad track and apply chorus effect.
2	L2-19	Duplicate a MIDI clip and transpose it up 12 semitones.
2	L2-20	Create a bass track and apply compression.
3	L3-01	Create a minimal techno drum pattern across 2 tracks, looped for 8 bars.
3	L3-02	Build a layered EDM drop with drums, bass, and synth lead across 3 tracks.
3	L3-03	Compose a 12-bar progression in A minor with chord movement and bassline.
3	L3-04	Create a full hip-hop beat with drums and sampled melody over 8 bars.
3	L3-05	Generate an ambient soundscape using 3 evolving pad tracks with reverb.
3	L3-06	Construct a house groove at 124 BPM with drums, bass, and chord stabs.
3	L3-07	Build a techno loop with filter cutoff automation increasing over time.
3	L3-08	Create an 8-bar pop chord progression with piano and layered synth pads.
3	L3-09	Generate a breakdown section with reduced drums over 16 bars.
3	L3-10	Create a multi-scene arrangement with intro, build, and drop sections.
3	L3-11	Build a progressive house loop with evolving automation over 8 bars.
3	L3-12	Create a lo-fi hip-hop beat with vinyl texture and swing rhythm.
3	L3-13	Generate a cinematic ambient piece with layered drones and reverb tails.
3	L3-14	Construct a trap beat with hi-hat rolls, 808 bass, and snare variations.
3	L3-15	Create a techno arrangement with kick-driven structure and filtering build.
3	L3-16	Build a melodic EDM lead over chord progression with sidechain compression.
3	L3-17	Generate a 16-bar evolving pad sequence with gradual harmonic changes.
3	L3-18	Create a funk-inspired groove with bassline and syncopated drums.
3	L3-19	Construct a full arrangement with intro, verse, chorus, and outro scenes.
3	L3-20	Build a layered electronic track with automation, effects, and transitions.